







Exo 1: Hospital management system

first attributes

Patient:

+ id: int
+ name: String
+ Password: String
+ email: String

Doctor:

+ id: int
+ name: String
+ Password: String
+ Speciality: String

Organ

+ id: int
+ name: String
+ Password: String
+ Role: String

then methods

Patient:

+ Post Problem()
+ make Payment()

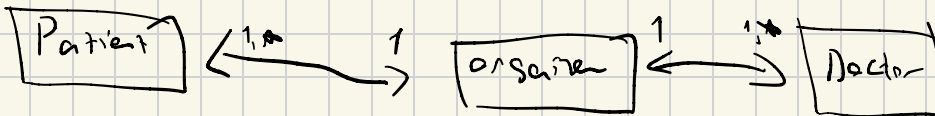
Doctor

+ read Problem()
+ Write Prescription()

Organ

+ Consult Doctor()
+ Send Prescription()
+ Format Payment()
+ read Problem()

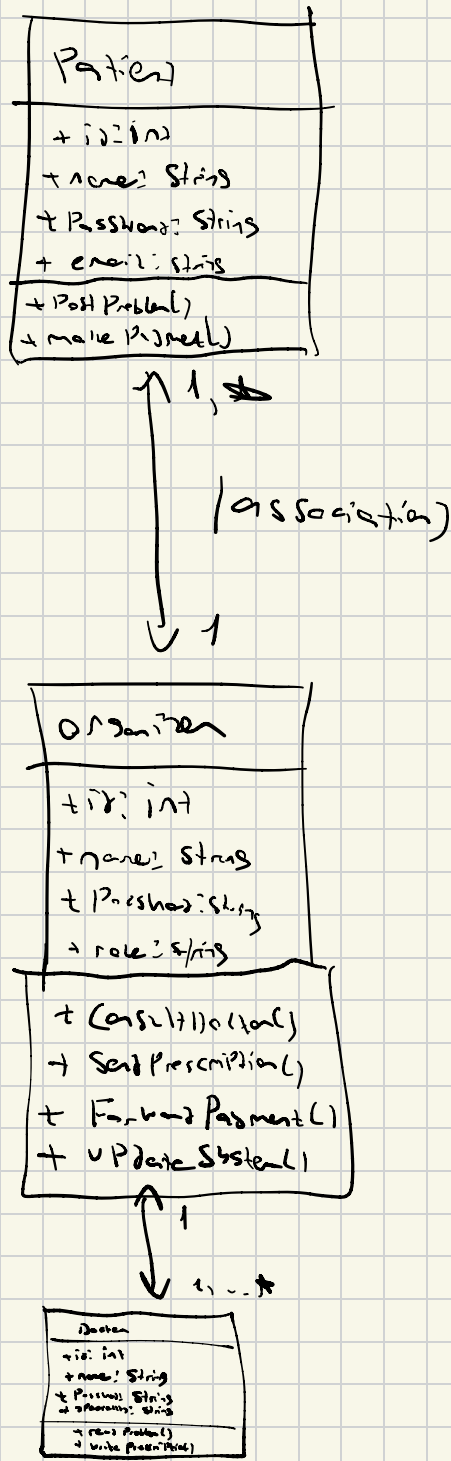
Now relationships



Patient can communicate organ, and organ can communicate to Doctor

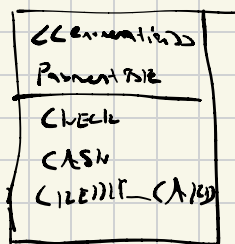
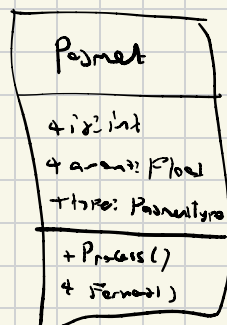
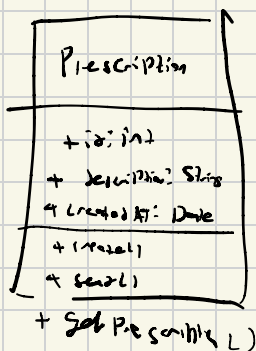
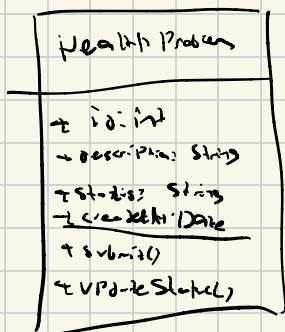
the same for organ and doctor (kin b-? latence, kin latence ok on the gate)

Class Diagram

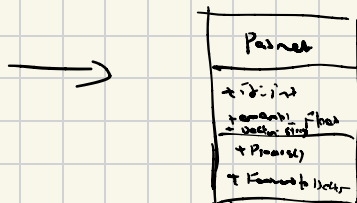
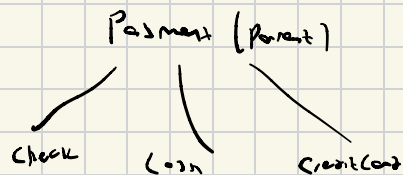


Task 2

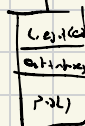
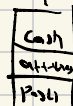
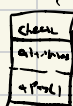
we add 3 new classes



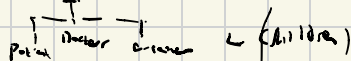
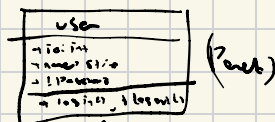
or, we can use inheritance for Payment



(inheritance)



Patient, Doctor, Organ, this sort then has its own P-classes, so we can create User Parent class



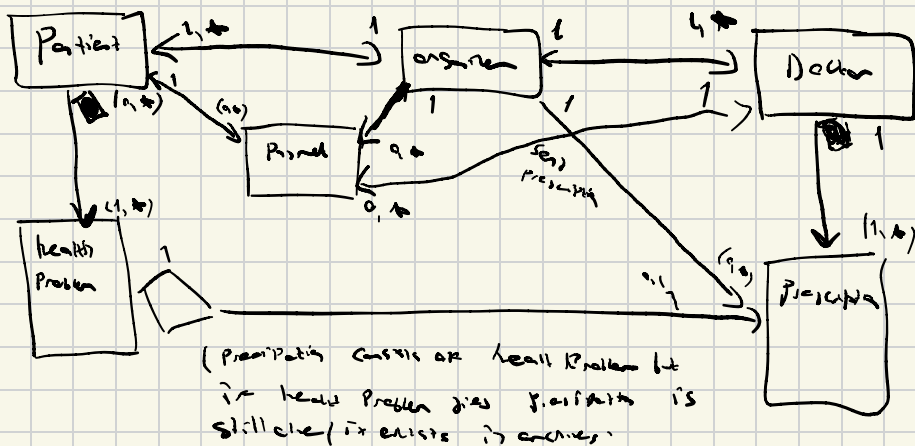
(Inheritance on class means Payment has methods + attributes + has its own methods + attributes (inheritance))

Composition

Patient  → healthProblem (if Patient dies, healthProblem dies as well)
 whole → part

Doctor  → Prescription (if doctor dies, prescription dies as well)

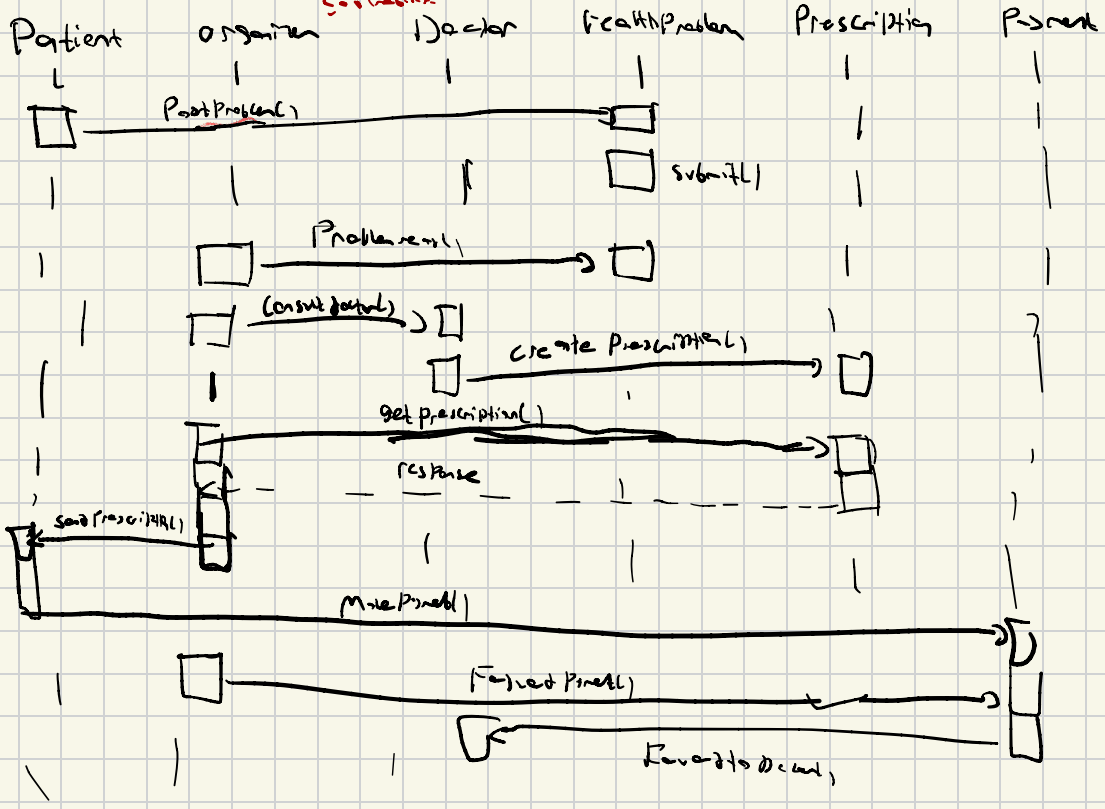
Relationships



Patient Person bilin, tann. Patient Person yollama arsa Person gair. yor
 arsa Person bilan Person has. Doktor arsa obru bilan, Person
 Doktor arsa obru. Doktor arsa obru. Doktor arsa obru. Doktor arsa obru.
 Doktor arsa obru. Doktor arsa obru. Doktor arsa obru. Doktor arsa obru.
 Doktor arsa obru. Doktor arsa obru. Doktor arsa obru. Doktor arsa obru.

Sequence Diagram

Not/ok mesajları için bu kadar ne diye sorulmuş olabilir.
 İki sınıfın bir arada çalışması için bir class başka bir class methodu
 çağırabilir



Sequence Diagram A → B denir A B si kordin
 denir Pirel Dektan Girmis ve ödegir ilenir.

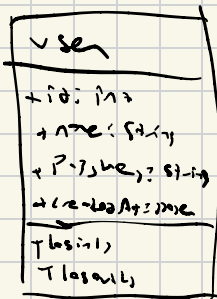
A → B = A B'ye mesaj gönderir

Self message Tık Diktir.

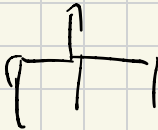
Pirel D-ktan ödegir gönderir

Task 3

1) we add user superclass

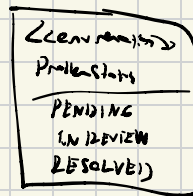
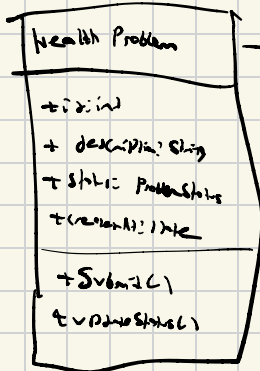


(Parent)

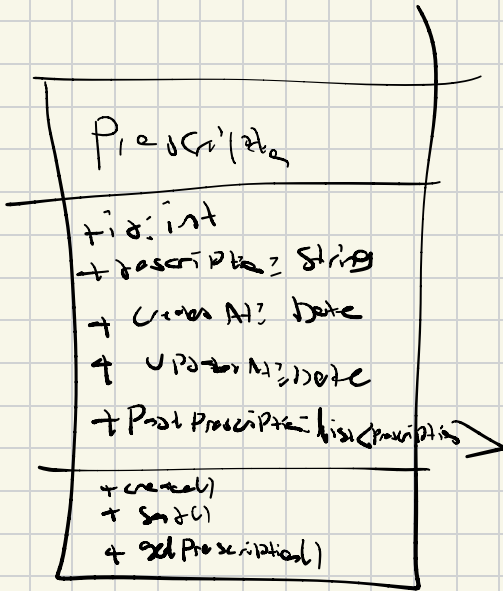


Patient Doctor Nurse (child)

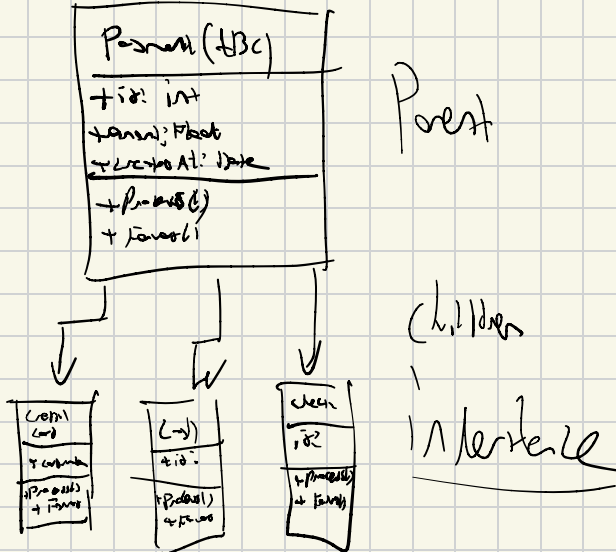
2) Adding status functions to Health Problem



3) we add time stamp and Post Prescription

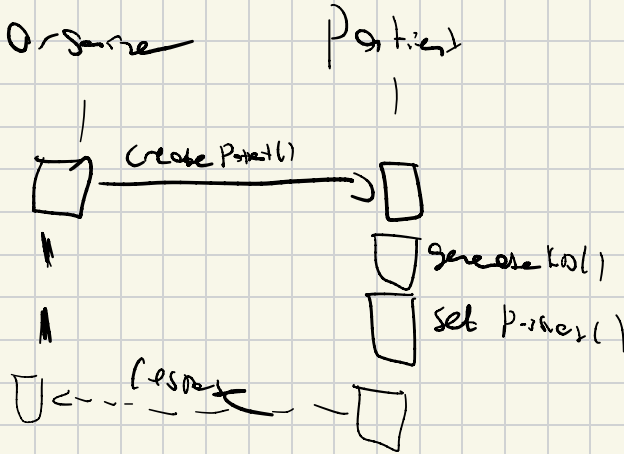


4) Parent Strategy - Polymorphism

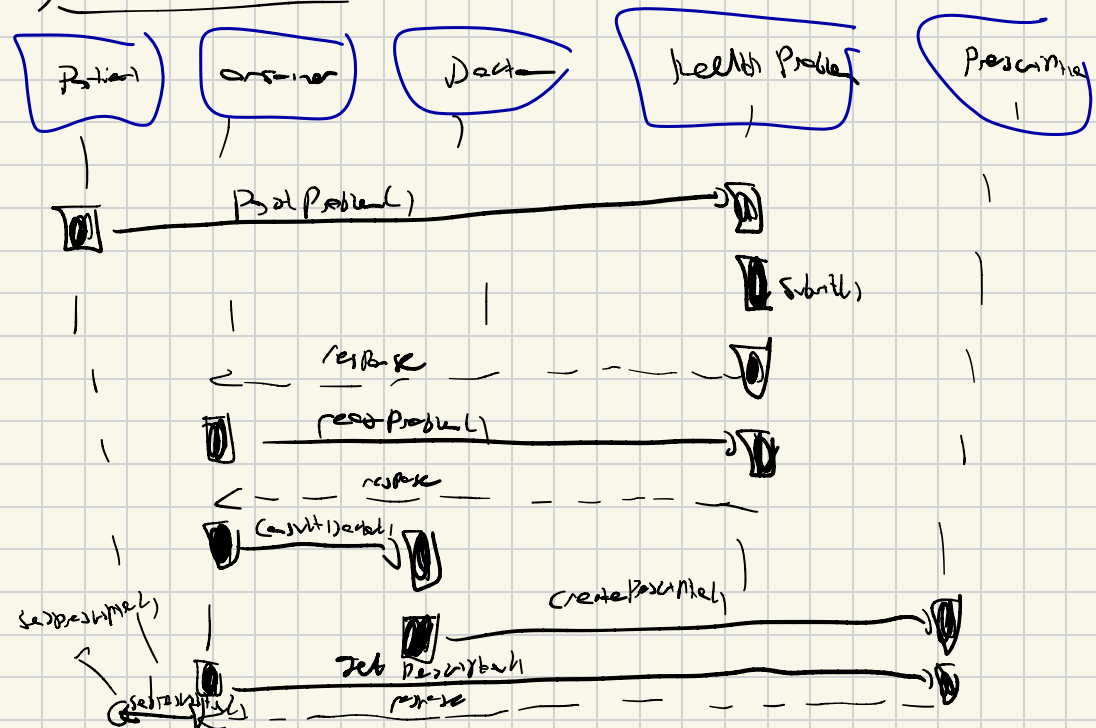


System Service Diagram For these scenarios

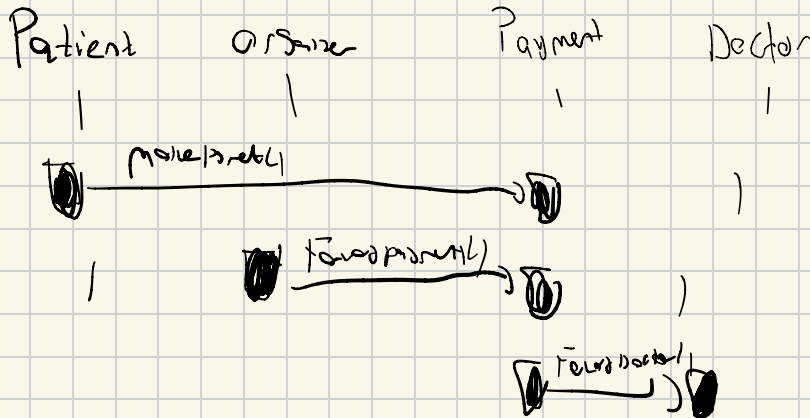
1) Patient registration



2) Consultation Plan



2) Payment Processing



1. Bei Single Action, create private forwardPayment method
 2. Since message yellow, forwardPayment method inside
 the Payment (forwardDoctor to Doctor) is not
 necessary.

1. How would you handle a patient consulting multiple doctors?

Add 1..* multiplicity to the Organizer → Doctor relationship. A single HealthProblem can receive multiple Prescriptions, one from each doctor consulted.

2. Should prescriptions be reusable? How would you model that?

Yes. Add a pastPrescriptions:

List<Prescription> attribute to the Prescription class. Doctors can reference previous prescriptions and reuse them when appropriate, without creating a new one from scratch.

3. What design pattern fits the payment system?

The Strategy Pattern. Payment is an abstract class, and Check, Cash, and CreditCard are concrete subclasses that each override the `process()` method differently. The payment type can be swapped at runtime without changing the surrounding code.

4. How do you ensure data integrity when payments are processed?

Add a `status` attribute to the Payment class with values PENDING, SUCCESS, and FAILED. Every payment starts as PENDING. If processing succeeds it becomes SUCCESS, if it fails it becomes FAILED. This ensures every transaction is tracked and no money is lost silently.

Question 2

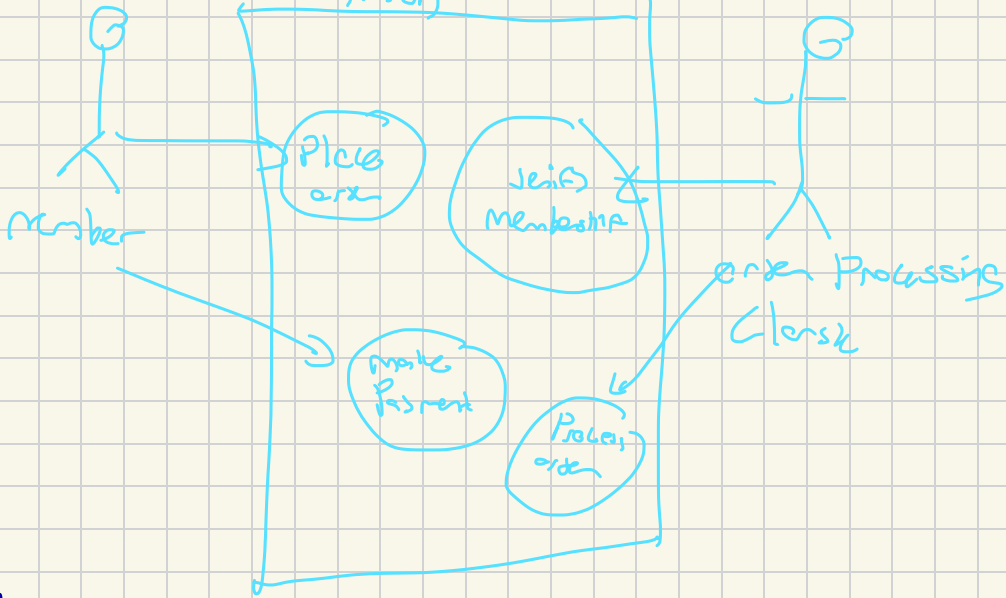
ABCD new

ds

1.2

use case diagram

ABCD



1.2

level 2

Use case diagram'da aktörler temel işlerine bağlanır, temel işler yapılırken otomatik tetiklenen adımlar include veya extend ile gösterilir.

nele sipariş

85

unavailable sipariş etme hanel sipariş

eğer, unavailable olursa

use case diagram

Arşiv

Yeni üye

onboard m → End
hne use case

hne (yeni üye place
order de
hne 60
öğrenci sınıfı)

interview
sınıfı

member

member

Collection
Department
Clerk

Order Processing
Class

Order Processing
Class

include --->
Kılder --->

Not: verisi ilişkisi iki kez olursa her Process
her işte

Not: nele sipariş verile

Stokta ne m işte

60 kişiye o stokta

belirli unavailable item'din ana baktıkça Place'de.

Process de bakıyor, Stokta ne m işte

Royal Member sipariş veriyor (Place Order). Sistem
bakıyor: "Bu ürün stokta yok ama bu adam Royal üye, o
zaman stokta olmayan ürünü de sipariş edebilir." → Order
Unavailable Items otomatik tetikleniyor.

Tam olarak öyle! Mükemmel özetledin.

Düz çizgiler (normal oklar) = "Bu aktör bu işe katılıyor"

- Member → Place Order = "Member sipariş veriyor"
- Collection Dept Clerk → Make Payment = "Clerk ödeme alıyor"

Kesikli oklar = Use case'ler arası ilişkiler, iki tipi var:
<<include>> (kesikli ok) = "Bu olunca şu her zaman olur"

- Process Order → Make Payment = "Sipariş işlenince ödeme **mutlaka** alınır"
- Process Order → Print Invoice = "Sipariş işlenince fatura **mutlaka** basılır"

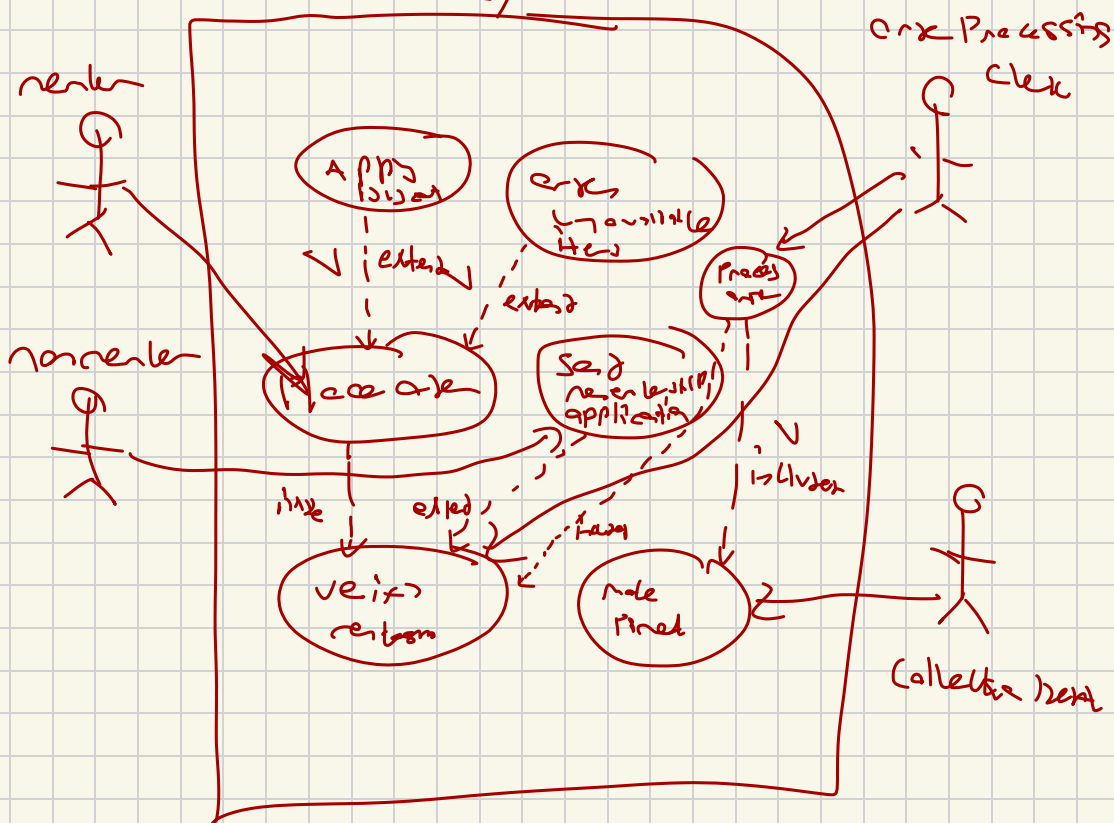
<<extend>> (kesikli ok) = "Bu olunca şu **bazen** olur"

- Apply Discount → Place Order = "Sipariş verilirken indirim **bazen** uygulanır (sadece Royal)"
- Order Unavailable Items → Place Order = "Sipariş verilirken stokta olmayan ürün **bazen** istenir"

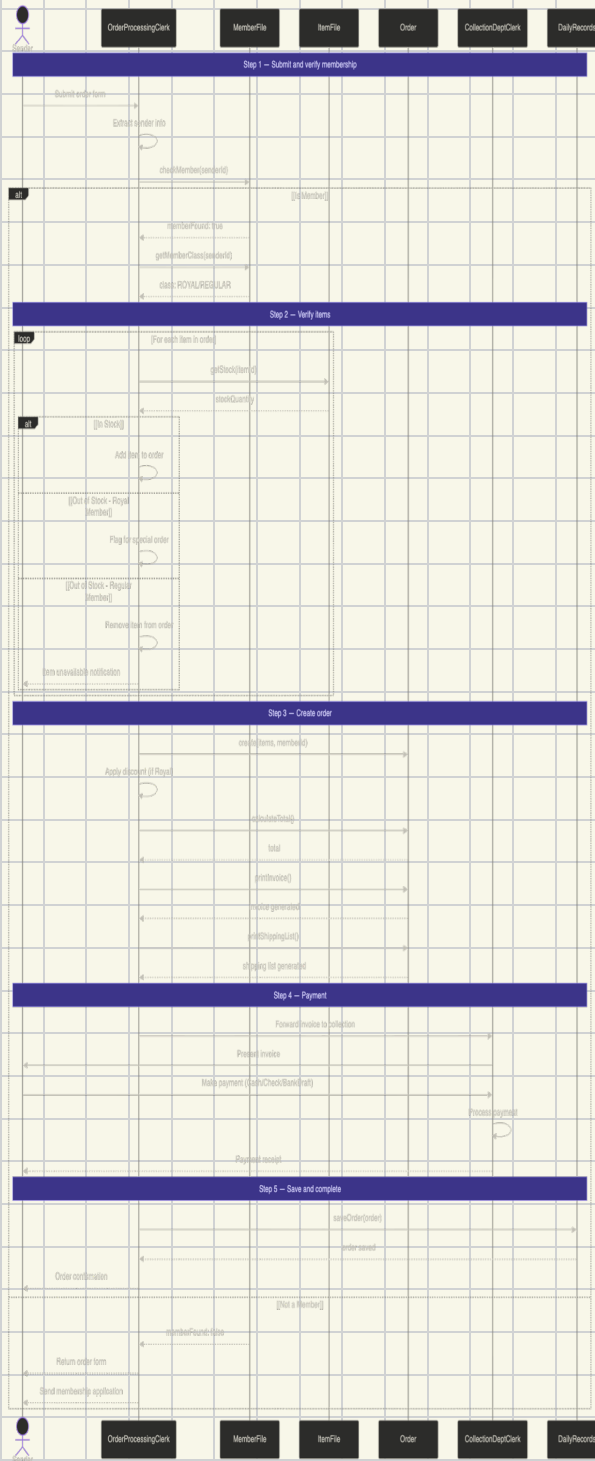
Kısaca:

- Düz = kim ne yapıyor
- Kesikli include = ne zaman ne **zorunlu**
- Kesikli extend = ne zaman ne **olabilir**

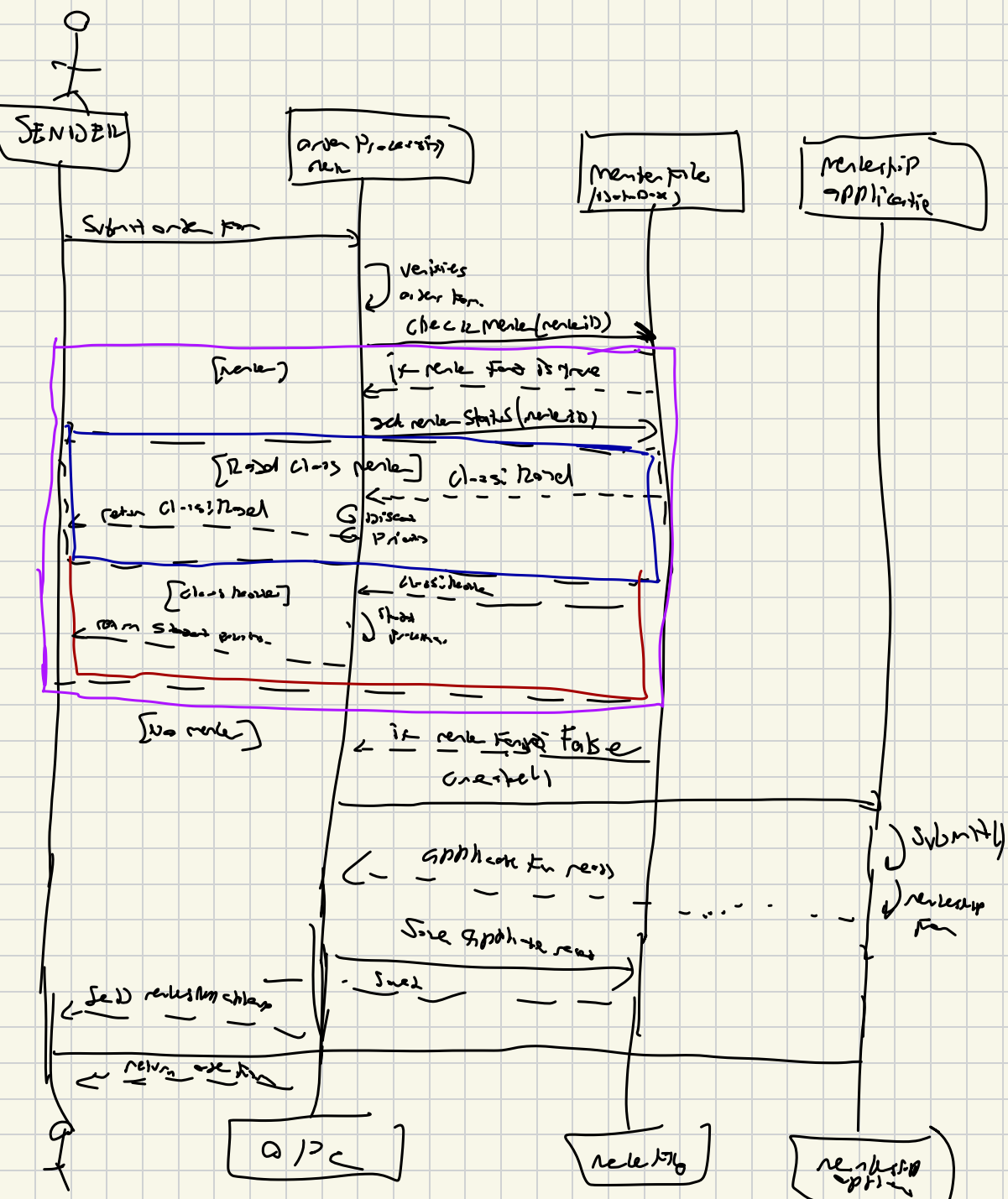
Platzhalter für die weiteren Aktivitäten



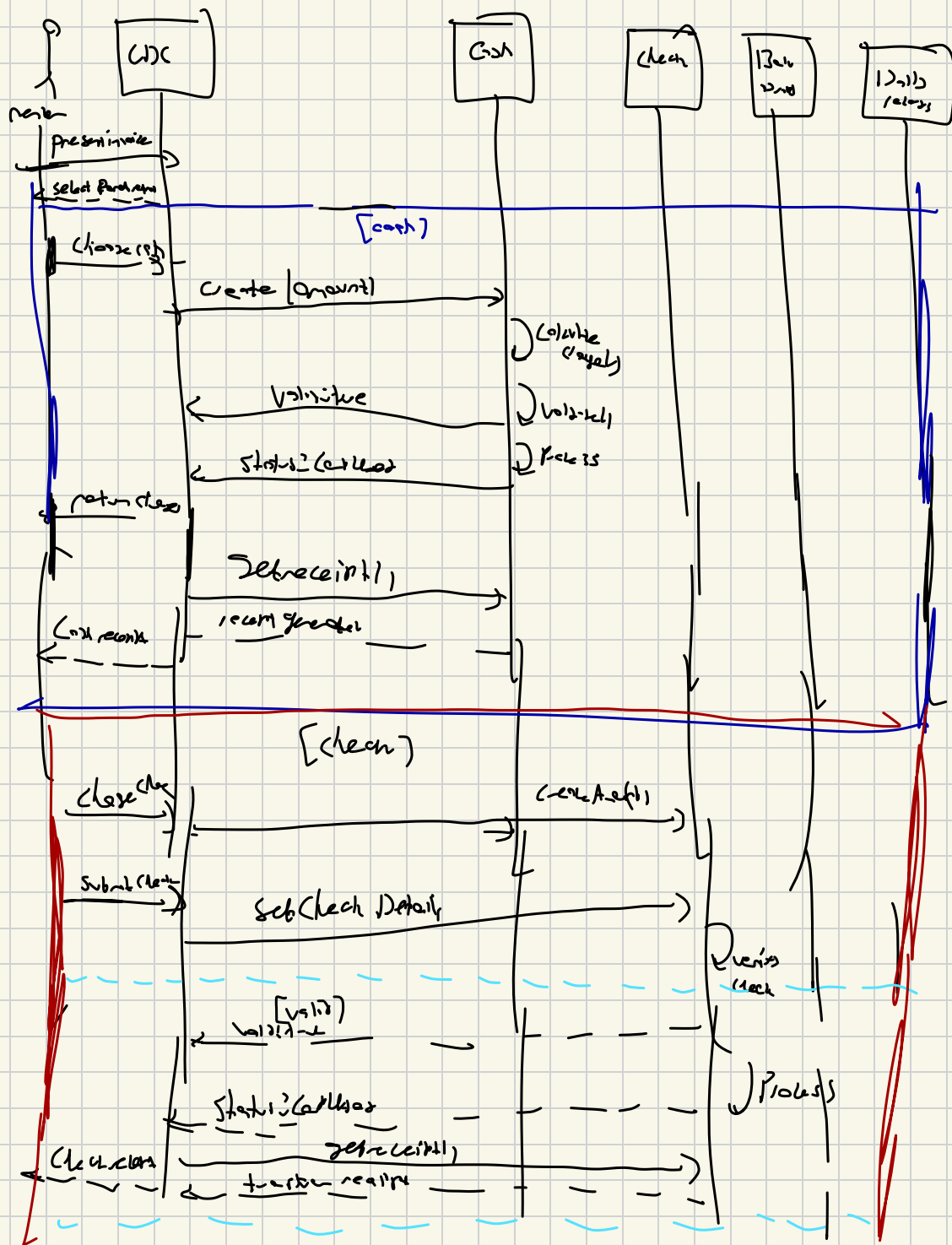
level 3



3 2 (membership verification)



33



[Front]

variable

Receptor

[Back Diagram]

Change Path
Name

code bank

Switch Path
Name

various details

not (extra)

extra: false

is valid

id: 1000

ring
work

[Extra] like

variable

Static: Carriers

get receiver

receipt + scores

Bank
name: sun

[Extra: Karas]

variable (many)

Refers

Reels

Expenses

entrepreneur

to grow

VP of Finance

VP of Sales

Chief Executive

to grow

CEO

COO

Mark

Ben

John

OrderProcessingClerk

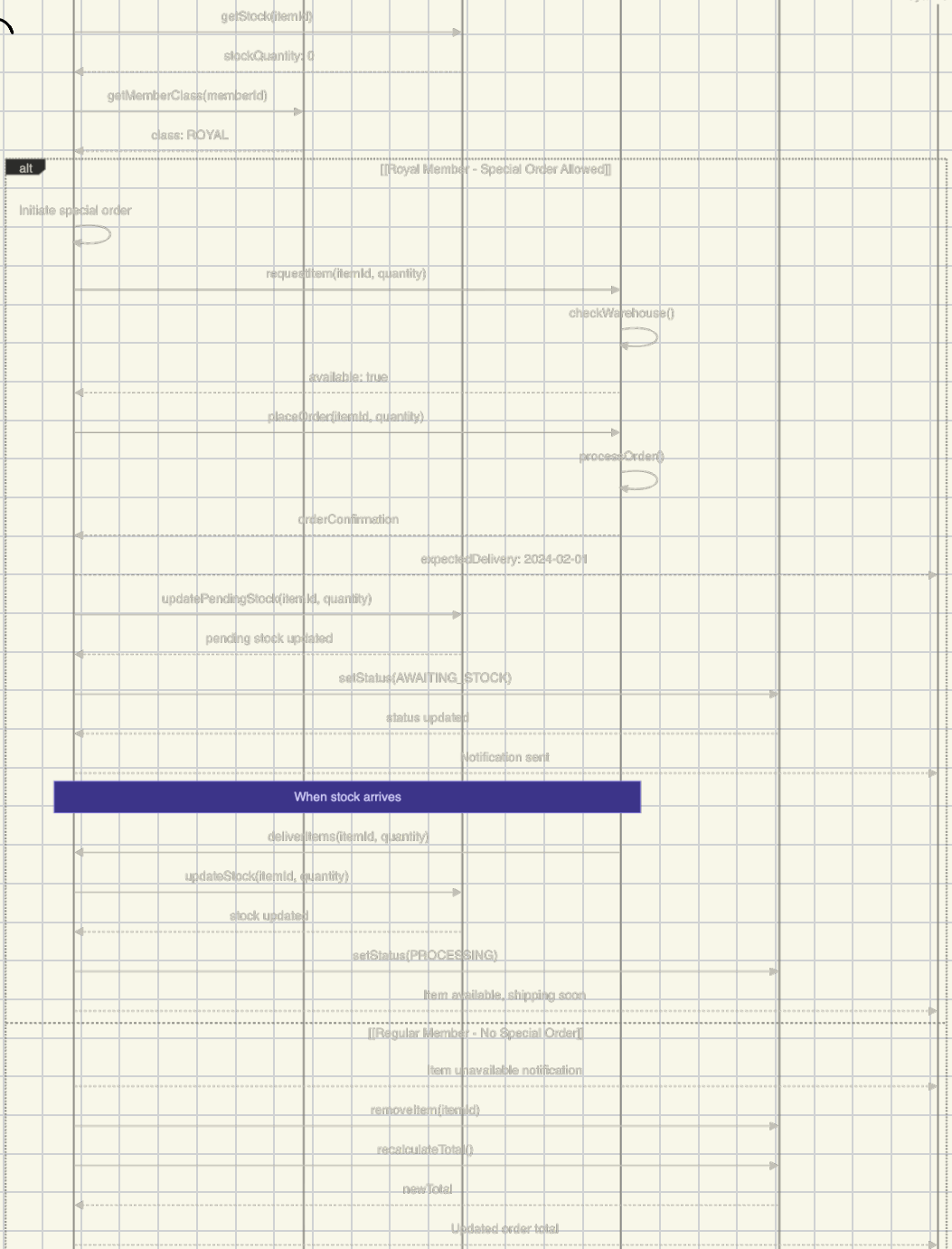
MemberFile

ItemFile

Supplier

Order

RoyalMember



OrderProcessingClerk

MemberFile

ItemFile

Supplier

Order

RoyalMember